

VendorPulse — Why Each Component Is Required

A justification reference for handling client change requests

*Purpose: when the client proposes removing or swapping a component, this gives you the rationale to push back with confidence — or to agree where it genuinely is flexible. For each component: what it does, why it's required, what breaks if it's removed, and the response to give. The two items flagged for deeper discussion — the **relational database** and the **MAF SDK** — have their own sections.*

How to use it: lead with the one-line justification, then escalate to "what breaks" only if pushed. Where something is genuinely negotiable, say so — it buys credibility for the parts that aren't.

Quick reference

Component	Required?	One-line justification	If the client wants to cut/change it
Relational DB (PostgreSQL)	Required	Transactional state, integrity, queryable audit — storage can't do these	See §1. Azure SQL is an acceptable swap; Blob/flat files are not
MAF SDK (agent layer)	Required (recommended)	Platform-provided approval, safety, tracing = less hand-rolled code for Shell to review	See §2. Reverting to self-built increases review burden
Front Door + App Gateway + WAF	Required (WAF non-negotiable)	One hardened, inspected way in; no public backend	Can simplify to a single edge if internal-only — see §3
Two-VM split (App / Backend)	Recommended	Separates the user-facing tier from data/integration	Can collapse to one VM to save cost, with reduced isolation
Entra OIDC + RBAC	Required	Shell identity, server-side authorization	Not negotiable — it's Shell's own standard
Approval gate (HITL)	Required	The core governance control; AI never acts alone	Not negotiable — it's the product's safety model
WorkflowEngine (12-state)	Required	Deterministic, auditable process; no skipped steps	Not negotiable — it's the deterministic core
Key Vault + Managed Identity	Required	No secrets in code; Shell compliance	Not negotiable
Blob Storage	Required (for files)	Transcripts/minutes; immutable records	Keep — but it is not a database substitute
App Insights + OTel	Required	Observability + immutable audit trail	Not negotiable for a governed AI system
Egress proxy	Required	Shell's controlled, logged outbound	Shell network policy — not ours to remove
Azure AI Foundry in-tenant	Required	Data residency — AI runs inside Shell	Not negotiable for the data-residency story

1. The relational database (Azure PostgreSQL) — required

Short answer to give: "Yes, an RDBMS is required. I'll walk through the rationale, but in one line: VendorPulse's data is transactional, related, queried, and audited — and object storage or flat files can't do any of those safely."

What it does: holds workflow state, user data, approval records, and audit logs — the system's source of truth.

Why it's required — four things only a database gives you:

- Transactions and concurrency.** The 12-state workflow and the approval gate must update state atomically, and multiple coordinators will act at once. A database gives row-level locking and ACID transactions so two simultaneous actions can't corrupt or lose state. Files/Blob are last-write-wins — one update silently overwrites another.

- Integrity and relationships.** Requests, approvals, attendees, and audit records are related. The database enforces those relationships (foreign keys, constraints) so the data can't drift into an invalid shape. Storage has no concept of this.

- Querying and reporting.** Dashboards and status views ("everything pending approval", "this sponsor's history") are single queries against indexed columns. Without a database you'd load and scan every record in application code — slow and unscalable.

- A queryable, tamper-evident audit trail.** Approval and audit records must be searchable and provable for IRM. A database makes the audit indexed and queryable; combined with the immutable Log Analytics mirror, it satisfies the compliance ask.

What breaks if it's removed: lost or corrupted state under concurrent use; no referential integrity; no reporting without scanning everything; a weak, hard-to-search audit trail. For a governance product these are not acceptable.

Where we can flex (say this to show reasonableness):

- Azure SQL Database is an acceptable alternative** if that's Shell's data-platform standard — same guarantees, different engine. The application accesses storage through a single repository layer, so switching engines is a contained change, not a rewrite.

- Cost is addressable without dropping the DB:** a serverless (Azure SQL) or Burstable (PostgreSQL) tier scales down between cycles.

Where we cannot flex: replacing the database with **Blob Storage, flat files, or a document/NoSQL store** for the core state. Blob is the right home for *files* (transcripts, minutes) and we already use it there — but it is not a transactional system of record.

One-liner: "Blob is for files; the workflow state, approvals and audit need a database. We can align on Postgres vs Azure SQL to your standard, and tune the tier for cost — but dropping the relational store would reintroduce concurrency, integrity and reporting problems we deliberately engineered out."

2. The MAF SDK (agent layer) — required, vs. reverting to self-implemented agents

Short answer to give: "We'd keep the Microsoft Agent Framework. Going back to hand-rolled agents means *more* custom, security-sensitive code for Shell to review and weaker built-in controls — the opposite of what helps us through governance."

What it does: the MAF SDK is how the AI Service's agents are built and run. It provides the agent loop, structured tool-calling, a built-in human-approval mode, content-safety integration, and on-by-default tracing.

Why it's required (the honest version):

- It shrinks the code Shell has to security-review.** Approval (HITL), content filtering, and tracing come from the Microsoft-maintained framework instead of being hand-written by us. Less bespoke, security-sensitive code = a smaller, faster code-security review (an IRM concern).

- It's a Microsoft-supported, GA framework** running on Foundry — not a bespoke tool loop we have to justify and maintain ourselves.

Consistency. One structure across every agent, instead of six bespoke call patterns.

It keeps the managed-hosting door open. The same agent code can later run as a Foundry Hosted Agent (managed compute) with no rewrite — a future option we get for free by choosing MAF.

Being straight about scope (use this, it builds credibility): most of our AI features are essentially *single structured calls* — "given this context, draft this text" — not heavy autonomous reasoning, and we don't force those into an agent loop. So we're not adopting MAF because we need autonomous agents; we're adopting it for the **governance controls and the supported, consistent structure**.

What reverting to self-implemented agents would cost:

We'd hand-roll the approval gate, content-safety hooks, and tracing ourselves — **more custom code in the security-sensitive path**, exactly what Shell's review scrutinises most.

We'd lose the platform-provided controls and the Microsoft support story.

We'd close off the Foundry Hosted Agents upgrade path.

Net: more effort, more review burden, a weaker governance narrative — a bad trade for a compliance-driven product.

Where we can flex: the model provider stays abstracted (config-driven), and the simplest single-shot calls can remain simple inside the framework. We are not over-engineering every feature into an agent.

One-liner: "MAF isn't force-fit — we use it for the controls and the supported structure, not because everything needs autonomous reasoning. Reverting to self-built agents would put more security-sensitive code in front of your reviewers and remove controls Microsoft already gives us. That's why we'd keep it."

3. The rest of the stack — why each is required

Edge: Front Door + Application Gateway + WAF. The single hardened, inspected way in — TLS termination, OWASP rules, origin-lock so no VM is reachable directly, and load balancing. *If removed:* the backend is exposed and unprotected against common web attacks. *Where we can flex:* if access is strictly internal, the global Front Door layer can be dropped and a single **Application Gateway + WAF** used instead — but the **WAF is non-negotiable**.

Two-VM split (App Server / Backend Services). Separates the user-facing tier (auth, workflow, UI) from the data and integration tier (database, Graph, AI). *If removed:* a single VM works but couples the tiers and reduces isolation. *Where we can flex:* the two VMs can be collapsed into one to save cost in lower environments, accepting reduced separation — a reasonable trade to discuss.

Entra OIDC + RBAC. Users sign in with Shell identity; their role is carried in the token and enforced server-side. *If removed:* no governed access control. *Not negotiable* — it's Shell's own identity standard.

Approval gate (HITL). Every AI output is a draft held until a human approves; the workflow blocks until then. *If removed:* the AI could drive actions unreviewed — the one thing the product exists to prevent. *Not negotiable*.

WorkflowEngine (12-state). The deterministic, forward-only state machine that drives every request. *If removed:* the process could be skipped or reordered. *Not negotiable* — it is the deterministic core.

Key Vault + Managed Identity. All secrets (LLM key, Graph cert, JWT keys) fetched at runtime via a password-less Azure identity. *If removed:* secrets end up in code/config — a direct compliance failure. *Not negotiable*.

Blob Storage. Stores transcripts and minutes, immutable once written. *If removed:* nowhere to keep large file artifacts. *Keep* — but note it is for files, not the database.

App Insights + Log Analytics (OpenTelemetry). Observability and the immutable audit trail of every action. *If removed:* no operational visibility and no provable audit. *Not negotiable* for a governed AI system.

Egress proxy. The single, governed, logged outbound path for Graph and Foundry. *If removed:* uncontrolled outbound traffic. *Not ours to remove* — it's Shell's network policy.

Azure AI Foundry in-tenant (GPT-4o). The model runs inside Shell's own tenant, so data sent to it never leaves the boundary. *If removed / replaced with a public API: breaks the data-residency story. Not negotiable* for that reason.

What we're genuinely flexible on (lead with this to build trust)

- bullet **Postgres vs Azure SQL** — align to Shell's data-platform standard.
- bullet **Database tier** — serverless/burstable to control cost between cycles.
- bullet **One edge vs two** — drop global Front Door if access is internal-only (keep the WAF).
- bullet **One VM vs two** — collapse the tiers in non-prod to save cost.
- bullet **Exact GPT model** — config-driven; we standardise on a GA model and can re-evaluate.

What we will not compromise on (and why)

- bullet **A relational database** for state and audit (integrity, concurrency, reporting).
- bullet **The human-approval gate** and the **deterministic WorkflowEngine** (the governance core).
- bullet **Key Vault + Managed Identity, Entra OIDC + RBAC, the WAF, OTel audit, and in-tenant Foundry** (Shell's own compliance and data-residency requirements).

Closing line: "We've separated what's genuinely a preference — the database engine, the tier, how many VMs — from what protects you: the approval gate, the deterministic core, identity, secrets, audit, and keeping the AI inside your tenant. We'll happily flex on the first list; the second list is what makes this pass your own IRM controls."